Assignment 7

Name: Riddhi Agarwal
Roll Number: 2024CSB046

Problem 1

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <time.h>

#define V 10
#define INF INT_MAX

struct AdjListNode {
    int dest;
    int weight;
    struct AdjListNode* next;
};

struct AdjList {
    struct AdjListNode* head;
};

struct MinHeapNode {
    int v;
    int dist;
};

struct MinHeap {
    int size;
    int capacity;
    int* pos;
    struct MinHeapNode** array;
};

struct AdjListNode* newAdjListNode(int dest, int weight) {
    struct AdjListNode* newNode = (struct AdjListNode*)malloc(sizeof(struct AdjListNode));
    newNode->dest = dest;
    newNode->weight = weight;
    newNode->next = NULL;
    return newNode;
}

```

```c
struct MinHeapNode* newMinHeapNode(int v, int dist) {
    struct MinHeapNode* minHeapNode = (struct MinHeapNode*)malloc(sizeof(struct MinHeapNode));
    minHeapNode->v = v;
    minHeapNode->dist = dist;
    return minHeapNode;
}

struct MinHeap* createMinHeap(int capacity) {
    struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct MinHeap));
    minHeap->pos = (int*)malloc(capacity * sizeof(int));
    minHeap->size = 0;
    minHeap->capacity = capacity;
    minHeap->array = (struct MinHeapNode**)malloc(capacity * sizeof(struct MinHeapNode*));
    return minHeap;
}

void swapMinHeapNode(struct MinHeapNode** a, struct MinHeapNode** b) {
    struct MinHeapNode* t = *a;
    *a = *b;
    *b = t;
}

void minHeapify(struct MinHeap* minHeap, int idx) {
    int smallest, left, right;
    smallest = idx;
    left = 2 * idx + 1;
    right = 2 * idx + 2;

    if (left < minHeap->size && minHeap->array[left]->dist < minHeap->array[smallest]->dist)
        smallest = left;

    if (right < minHeap->size && minHeap->array[right]->dist < minHeap->array[smallest]->dist)
        smallest = right;

    if (smallest != idx) {
        struct MinHeapNode* smallestNode = minHeap->array[smallest];
        struct MinHeapNode* idxNode = minHeap->array[idx];

        minHeap->pos[smallestNode->v] = idx;
        minHeap->pos[idxNode->v] = smallest;

        swapMinHeapNode(&minHeap->array[smallest], &minHeap->array[idx]);
        minHeapify(minHeap, smallest);
    }
```

```c
}

struct MinHeapNode* extractMin(struct MinHeap* minHeap) {
    if (minHeap->size == 0) return NULL;
    struct MinHeapNode* root = minHeap->array[0];
    struct MinHeapNode* lastNode = minHeap->array[minHeap->size - 1];
    minHeap->array[0] = lastNode;
    minHeap->pos[root->v] = minHeap->size - 1;
    minHeap->pos[lastNode->v] = 0;
    --minHeap->size;
    minHeapify(minHeap, 0);
    return root;
}

void decreaseKey(struct MinHeap* minHeap, int v, int dist) {
    int i = minHeap->pos[v];
    minHeap->array[i]->dist = dist;
    while (i && minHeap->array[i]->dist < minHeap->array[(i - 1) / 2]->dist) {
        minHeap->pos[minHeap->array[i]->v] = (i - 1) / 2;
        minHeap->pos[minHeap->array[(i - 1) / 2]->v] = i;
        swapMinHeapNode(&minHeap->array[i], &minHeap->array[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}

int isInMinHeap(struct MinHeap* minHeap, int v) {
    if (minHeap->pos[v] < minHeap->size) return 1;
    return 0;
}

void dijkstraAdjList(struct AdjList* adj, int src) {
    int dist[V];
    struct MinHeap* minHeap = createMinHeap(V);

    for (int v = 0; v < V; ++v) {
        dist[v] = INF;
        minHeap->array[v] = newMinHeapNode(v, dist[v]);
        minHeap->pos[v] = v;
    }

    dist[src] = 0;
    decreaseKey(minHeap, src, dist[src]);
    minHeap->size = V;

    while (minHeap->size != 0) {
        struct MinHeapNode* minHeapNode = extractMin(minHeap);
        int u = minHeapNode->v;
        struct AdjListNode* pCrawl = adj[u].head;
        while (pCrawl != NULL) {
```

```
            int v = pCrawl->dest;
            if (isInMinHeap(minHeap, v) && dist[u] != INF && pCrawl->weight + dist[u] < dist[v]) {
                dist[v] = dist[u] + pCrawl->weight;
                decreaseKey(minHeap, v, dist[v]);
            }
            pCrawl = pCrawl->next;
        }
    }
    printf("Vertex   Distance from Source %d\n", src);
    for (int i = 0; i < V; ++i) printf("%d \t\t %d\n", i, dist[i]);
}

void dijkstraAdjMatrix(int matrix[V][V], int src) {
    int dist[V];
    struct MinHeap* minHeap = createMinHeap(V);

    for (int v = 0; v < V; ++v) {
        dist[v] = INF;
        minHeap->array[v] = newMinHeapNode(v, dist[v]);
        minHeap->pos[v] = v;
    }

    dist[src] = 0;
    decreaseKey(minHeap, src, dist[src]);
    minHeap->size = V;

    while (minHeap->size != 0) {
        struct MinHeapNode* minHeapNode = extractMin(minHeap);
        int u = minHeapNode->v;

        for (int v = 0; v < V; v++) {
            if (matrix[u][v] && isInMinHeap(minHeap, v) && dist[u] != INF && matrix[u][v] + dist[u] < dist[v]) {
                dist[v] = dist[u] + matrix[u][v];
                decreaseKey(minHeap, v, dist[v]);
            }
        }
    }
    printf("Vertex   Distance from Source %d\n", src);
    for (int i = 0; i < V; ++i) printf("%d \t\t %d\n", i, dist[i]);
}

int main() {
    struct AdjList adjList[V];
    for (int i = 0; i < V; i++) adjList[i].head = NULL;
    int adjMatrix[V][V] = {0};

    int edges, u, v, w, src;
    printf("Enter number of edges: ");
    scanf("%d", &edges);
```

```c

    for (int i = 0; i < edges; i++) {
        printf("Enter edge (u v w) [0-9]: ");
        scanf("%d %d %d", &u, &v, &w);
        struct AdjListNode* newNode = newAdjListNode(v, w);
        newNode->next = adjList[u].head;
        adjList[u].head = newNode;
        adjMatrix[u][v] = w;
    }

    printf("Enter source vertex: ");
    scanf("%d", &src);

    clock_t start, end;

    printf("\n--- Scenario 1: Adjacency List + Min-Heap ---\n");
    start = clock();
    dijkstraAdjList(adjList, src);
    end = clock();
    printf("Execution Time: %f seconds\n", (double)(end - start) / CLOCKS_PER_SEC);

    printf("\n--- Scenario 2: Adjacency Matrix + Min-Heap ---\n");
    start = clock();
    dijkstraAdjMatrix(adjMatrix, src);
    end = clock();
    printf("Execution Time: %f seconds\n", (double)(end - start) / CLOCKS_PER_SEC);

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> ./a.exe
Enter number of edges: 14
Enter edge (u v w) [0-9]: 0 1 4
Enter edge (u v w) [0-9]: 0 7 8
Enter edge (u v w) [0-9]: 1 2 8
Enter edge (u v w) [0-9]: 1 7 11
Enter edge (u v w) [0-9]: 2 3 7
Enter edge (u v w) [0-9]: 2 8 2
Enter edge (u v w) [0-9]: 2 5 4
Enter edge (u v w) [0-9]: 3 4 9
Enter edge (u v w) [0-9]: 3 5 14
Enter edge (u v w) [0-9]: 4 5 10
Enter edge (u v w) [0-9]: 5 6 2
Enter edge (u v w) [0-9]: 6 7 1
Enter edge (u v w) [0-9]: 6 8 6
Enter edge (u v w) [0-9]: 7 8 7
Enter source vertex: 0

--- Scenario 1: Adjacency List + Min-Heap ---
Vertex   Distance from Source 0
0                0
1                4
2                12
3                19
4                28
5                16
6                18
7                8
8                14
9                2147483647
Execution Time: 0.003000 seconds

--- Scenario 2: Adjacency Matrix + Min-Heap ---
Vertex   Distance from Source 0
0                0
1                4
2                12
3                19
4                28
5                16
6                18
7                8
8                14
9                2147483647
Execution Time: 0.002000 seconds
```

Figure 1: Output for Problem 1(Sparse Graph)

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> gcc Ques1.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> ./a.exe
Enter number of edges: 30
Enter edge (u v w) [0-9]: 0 1 2
Enter edge (u v w) [0-9]: 0 2 5
Enter edge (u v w) [0-9]: 0 3 1
Enter edge (u v w) [0-9]: 0 4 6
Enter edge (u v w) [0-9]: 0 5 8
Enter edge (u v w) [0-9]: 1 2 3
Enter edge (u v w) [0-9]: 1 4 2
Enter edge (u v w) [0-9]: 1 6 7
Enter edge (u v w) [0-9]: 1 8 4
Enter edge (u v w) [0-9]: 2 3 1
Enter edge (u v w) [0-9]: 2 5 3
Enter edge (u v w) [0-9]: 2 7 6
Enter edge (u v w) [0-9]: 3 4 5
Enter edge (u v w) [0-9]: 3 6 2
Enter edge (u v w) [0-9]: 3 9 8
Enter edge (u v w) [0-9]: 4 5 1
Enter edge (u v w) [0-9]: 4 7 4
Enter edge (u v w) [0-9]: 4 8 3
Enter edge (u v w) [0-9]: 5 6 4
Enter edge (u v w) [0-9]: 5 8 2
Enter edge (u v w) [0-9]: 5 9 5
Enter edge (u v w) [0-9]: 6 7 3
Enter edge (u v w) [0-9]: 6 9 2
Enter edge (u v w) [0-9]: 7 8 1
Enter edge (u v w) [0-9]: 7 9 6
Enter edge (u v w) [0-9]: 8 9 2
Enter edge (u v w) [0-9]: 9 0 4
Enter edge (u v w) [0-9]: 9 1 5
Enter edge (u v w) [0-9]: 9 2 7
Enter edge (u v w) [0-9]: 8 0 3
Enter source vertex: 0
```

Figure 2: Input for Problem 1(Dense Graph)

```
--- Scenario 1: Adjacency List + Min-Heap ---
Vertex   Distance from Source 0
0                0
1                2
2                5
3                1
4                4
5                5
6                3
7                6
8                6
9                5
Execution Time: 0.002000 seconds

--- Scenario 2: Adjacency Matrix + Min-Heap ---
Vertex   Distance from Source 0
0                0
1                2
2                5
3                1
4                4
5                5
6                3
7                6
8                6
9                5
Execution Time: 0.003000 seconds
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7>
```

Figure 3: Output for Problem 1(Dense Graph)

Problem 2

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define V 10
#define INF INT_MAX

struct Node {
    int v, dist;
};

void dijkstra(int graph[V][V], int src) {
    int dist[V];
    int visited[V];

    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int min = INF, u = -1;

        for (int v = 0; v < V; v++) {
            if (!visited[v] && dist[v] <= min) {
                min = dist[v];
                u = v;
            }
        }

        if (u == -1) break;
        visited[u] = 1;

        for (int v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] != 0 && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("\n--- Dijkstra's Results (Source: %d) ---\n", src);
    printf("Vertex \t Distance from Source\n");
    for (int i = 0; i < V; i++) {
        if (dist[i] == INF)
```

```c
            printf("%d \t INF\n", i);
        else
            printf("%d \t %d\n", i, dist[i]);
    }
}

int main() {
    int graph[V][V] = {0};
    int edges, u, v, w, src;

    printf("--- Dijkstra Limitation Experiment ---\n");
    printf("Enter number of edges: ");
    scanf("%d", &edges);

    printf("Enter edges in format (u v w). \nNote: Use negative weights to create a negative cycle.\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d %d", &u, &v, &w);
        if (u < V && v < V) {
            graph[u][v] = w;
        }
    }

    printf("Enter source vertex (0-9): ");
    scanf("%d", &src);

    dijkstra(graph, src);

    printf("\nObservation: If you included a negative cycle, note that Dijkstra \n");
    printf("cannot find the true shortest path because it never revisits 'visited' nodes.\n");

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> gcc .\Ques2.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> ./a.exe
--- Dijkstra Limitation Experiment ---
Enter number of edges: 10
Enter edges in format (u v w).
Note: Use negative weights to create a negative cycle.
0 1 2
1 2 3
2 3 1
3 4 2
4 5 4
5 6 3
6 4 -10
6 7 2
7 8 1
8 9 5
Enter source vertex (0-9): 0

--- Dijkstra's Results (Source: 0) ---
Vertex   Distance from Source
0        0
1        2
2        5
3        6
4        8
5        12
6        15
7        17
8        18
9        23

Observation: If you included a negative cycle, note that Dijkstra
cannot find the true shortest path because it never revisits 'visited' nodes.
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7>
```

Figure 4: Output for Problem 2.

Problem 3

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <float.h>

typedef struct {
    double x, y;
} Point;

int compareX(const void* a, const void* b) {
    Point *p1 = (Point *)a, *p2 = (Point *)b;
    return (p1->x > p2->x) - (p1->x < p2->x);
}

int compareY(const void* a, const void* b) {
    Point *p1 = (Point *)a, *p2 = (Point *)b;
    return (p1->y > p2->y) - (p1->y < p2->y);
}

double dist(Point p1, Point p2) {
    return sqrt((p1.x - p2.x)*(p1.x - p2.x) + (p1.y - p2.y)*(p1.y - p2.y));
}

void findClosest1D(Point pts[], int n) {
    qsort(pts, n, sizeof(Point), compareX);
    double min_dist = DBL_MAX;
    Point p1, p2;

    for (int i = 0; i < n - 1; i++) {
        double d = fabs(pts[i].x - pts[i+1].x);
        if (d < min_dist) {
            min_dist = d;
            p1 = pts[i];
            p2 = pts[i+1];
        }
    }
    printf("Closest 1D Pair: (%.2f) and (%.2f) with distance %.2f\n", p1.x, p2.x, min_dist);
}

double bruteForce2D(Point pts[], int n, Point *bestP1, Point *bestP2) {
    double min = DBL_MAX;
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            double d = dist(pts[i], pts[j]);
```

```c
            if (d < min) {
                min = d;
                *bestP1 = pts[i];
                *bestP2 = pts[j];
            }
        }
    }
    return min;
}

void findClosest2D(Point pts[], int n) {
    Point p1, p2;
    double min_dist = bruteForce2D(pts, n, &p1, &p2);
    printf("Closest 2D Pair: (%.2f, %.2f) and (%.2f, %.2f) with distance %.2f\n",
            p1.x, p1.y, p2.x, p2.y, min_dist);
}

int main() {
    int n;
    printf("Enter number of points: ");
    scanf("%d", &n);

    Point *pts = (Point*)malloc(n * sizeof(Point));
    printf("Enter coordinates (x y) for each point:\n");
    for (int i = 0; i < n; i++) {
        scanf("%le %le", &pts[i].x, &pts[i].y);
    }

    printf("\n--- Results ---\n");
    findClosest1D(pts, n);

    findClosest2D(pts, n);

    free(pts);
    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> gcc .\Ques3.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> ./a.exe
Enter number of points: 5
Enter coordinates (x y) for each point:
1 2
4 6
7 8
8 9
12 1

--- Results ---
Closest 1D Pair: (7.00) and (8.00) with distance 1.00
Closest 2D Pair: (7.00, 8.00) and (8.00, 9.00) with distance 1.41
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7>
```

Figure 5: Output for Problem 3.

Problem 4

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define INF INT_MAX

struct Edge {
    int src, dest, weight;
};

struct Graph {
    int V, E;
    struct Edge* edge;
};

struct Graph* createGraph(int V, int E) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = V;
    graph->E = E;
    graph->edge = (struct Edge*)malloc(E * sizeof(struct Edge));
    return graph;
}

void BellmanFord(struct Graph* graph, int src) {
    int V = graph->V;
    int E = graph->E;
    int dist[V];

    for (int i = 0; i < V; i++)
        dist[i] = INF;
    dist[src] = 0;

    for (int i = 1; i <= V - 1; i++) {
        for (int j = 0; j < E; j++) {
            int u = graph->edge[j].src;
            int v = graph->edge[j].dest;
            int weight = graph->edge[j].weight;
            if (dist[u] != INF && dist[u] + weight < dist[v])
                dist[v] = dist[u] + weight;
        }
    }

    int hasNegativeCycle = 0;
    for (int i = 0; i < E; i++) {
        int u = graph->edge[i].src;
        int v = graph->edge[i].dest;
```

```c
        int weight = graph->edge[i].weight;
        if (dist[u] != INF && dist[u] + weight < dist[v]) {
            hasNegativeCycle = 1;
            break;
        }
    }

    if (hasNegativeCycle) {
        printf("\nGraph contains a negative weight cycle!\n");
        printf("Shortest path values are undefined as they can decrease indefinitely.\n");
    } else {
        printf("\nVertex   Distance from Source %d\n", src);
        for (int i = 0; i < V; i++) {
            if (dist[i] == INF)
                printf("%d \t\t INF\n", i);
            else
                printf("%d \t\t %d\n", i, dist[i]);
        }
    }
}

int main() {
    int v_count, e_count, source;

    printf("Enter number of vertices: ");
    scanf("%d", &v_count);
    printf("Enter number of edges: ");
    scanf("%d", &e_count);

    struct Graph* graph = createGraph(v_count, e_count);

    for (int i = 0; i < e_count; i++) {
        printf("Enter edge %d (src dest weight): ", i + 1);
        scanf("%d %d %d", &graph->edge[i].src, &graph->edge[i].dest, &graph->edge[i].weight);
    }

    printf("Enter source vertex: ");
    scanf("%d", &source);

    BellmanFord(graph, source);

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> gcc .\Ques4.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> ./a.exe
Enter number of vertices: 4
Enter number of edges: 4
Enter edge 1 (src dest weight): 0 1 10
Enter edge 2 (src dest weight): 0 2 15
Enter edge 3 (src dest weight): 1 3 5
Enter edge 4 (src dest weight): 2 3 -10
Enter source vertex: 0

Vertex   Distance from Source 0
0                0
1                10
2                15
3                5
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7>
```

Figure 6: Output for Problem 4(Negative Weights (No Cycle))

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> gcc .\Ques4.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7> ./a.exe
Enter number of vertices: 5
Enter number of edges: 5
Enter edge 1 (src dest weight): 0 1 4
Enter edge 2 (src dest weight): 1 2 -2
Enter edge 3 (src dest weight): 2 3 3
Enter edge 4 (src dest weight): 3 1 -5
Enter edge 5 (src dest weight): 0 4 5
Enter source vertex: 0

Graph contains a negative weight cycle!
Shortest path values are undefined as they can decrease indefinitely.
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment7>
```

Figure 7: Output for Problem 4(Negative Weight Cycle Detected)